

1.5 Phân tích và đánh giá giải thuật

1.5.1 Khái niệm thuật toán

1.5.2 Thời gian chạy của thuật toán

1.5.2.1 Phương pháp thực nghiệm

1.5.2.2 Phương pháp lý thuyết

1.5.3 Các giả thuyết của phương pháp phân tích lý thuyết

1.5.4 Giả mã

1.5.5 Các phép toán cơ sở và cách xác định

1.5.6 Ký hiệu Big O

1.5.7 Đánh giá độ phức tạp của thuật toán

1.5.8 Các quy tắc

1.5.9 Đánh giá thuật toán

1. Khái niệm Thuật toán



Thuật toán là một tập các chỉ dẫn hoặc logic, được viết theo thứ tự, để hoàn thành một nhiệm vụ được xác định trước.

Thuật toán không phải là một đoạn code hoặc một chương trình hoàn chỉnh, mà nó chỉ là phần logic cốt lõi (lời giải) của một bài toán, nó có thể được diễn tả bằng ngôn ngữ giả mã hoặc sử dụng lưu đồ

Các tính chất:

1. **Input:** Không có hoặc có nhiều giá trị input
2. **Output:** Có ít nhất 1 giá trị
3. **Tính rõ ràng:** Mọi bước thực hiện của thuật toán phải rõ ràng và dễ dàng xác định
4. **Tính hữu hạn:** thuật toán phải có hữu hạn số bước thực hiện.
5. **Tính chính xác:** Mỗi bước thực hiện của thuật toán phải tạo ra một kết quả đúng chính xác.

1.5.

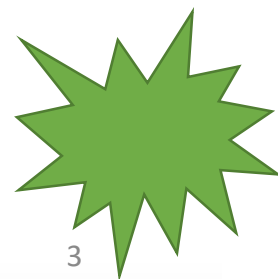
1. Khái niệm Thuật toán



Thuật toán là một qui trình thực hiện từng bước, từng bước giải quyết một vấn đề trong một khoảng thời gian hữu hạn.

Hai chủ đề cần phân tích

- Bộ nhớ: xác định tổng dung lượng bộ nhớ cần thiết để lưu trữ toàn bộ dữ liệu đầu vào, trung gian và kết quả đầu ra.
- Thời gian chạy của thuật toán



1.5.

2. Thời gian chạy của thuật toán

Thời gian chạy của thuật được đặc trưng bởi kích thước của dữ liệu đầu vào

Thời gian chạy của thuật toán được đánh giá trong 3 trường hợp: xấu nhất, trung bình và tốt nhất.

Tập trung vào trường hợp xấu nhất.

Phương pháp đánh giá

1. Phương pháp thực nghiệm
2. Phương pháp phân tích lý thuyết

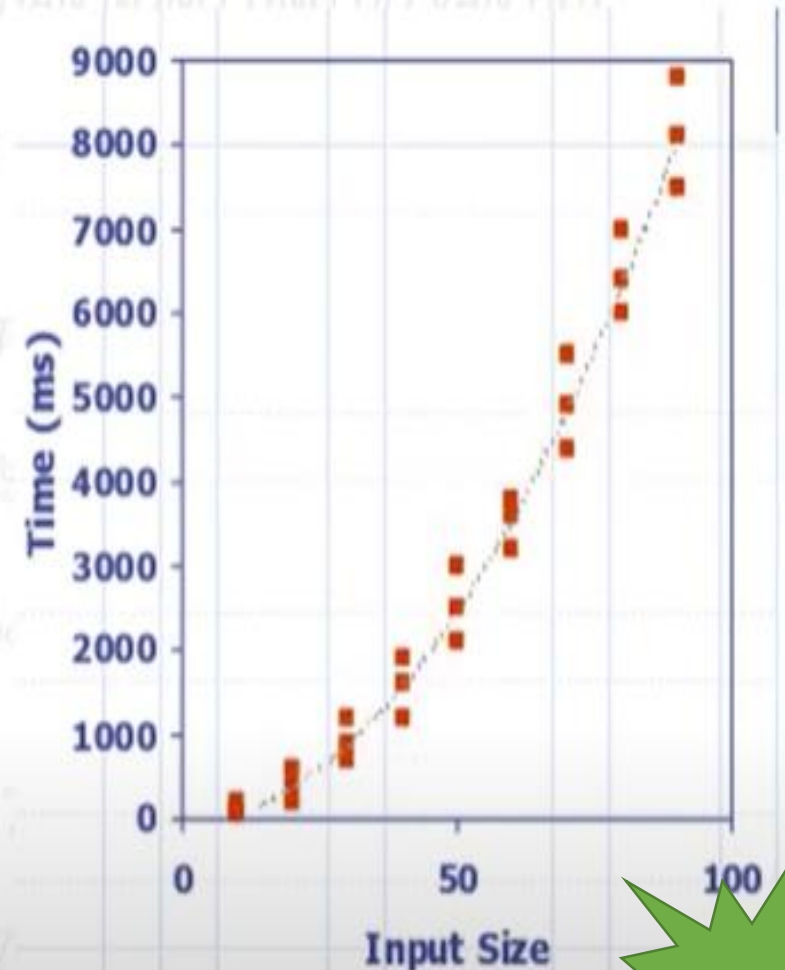


1.5.

2.1 Phương pháp thực nghiệm

Các bước thực hiện:

1. Viết một chương trình thể hiện thuật toán
2. Chạy chương trình với các bộ dữ liệu đầu vào khác nhau và lấy thời gian chạy của thuật toán.
3. Vẽ đồ thị biểu diễn kết quả



Ví dụ 1: Tính thời gian chạy của thuật toán

Chương trình cố định input

BT_04_01a.cpp

```
1 #include <iostream>
2 #include <stdlib.h>
3 using namespace std;
4
5 int main(){
6     system("color F0");
7     int n = 100;
8     for(int i = 0; i < n; i++)
9     {
10         int a = rand();
11         cout<< a<<" ";
12     }
13     return 0;
14 }
```

```
C:\Users\sev_user\Desktop\CodeCTDLGT_Duc\Code_CTDLGT_04\BT_04_01a.exe
41 18467 6334 26500 19169 15724 11478 29358 26962 ;
7 5436 32391 14604 3902 153 292 12382 17421 18716
26299 17035 9894 28703 23811 31322 30333 17673 4664
7 12859 8723 9741 27529 778 12316 3035 22190 1842
729 24370 15350 15006 31101 24393 3548 19629 12623
32439 24626 11323 5537 21538 16118 2082 22929 16541
.....
Process exited after 0.2252 seconds with return value 0
Press any key to continue . . .
```

Stt	Time (s)
1	0.2252
2	0.1735
3	0.1506
4	0.2352
5	0.2609
6	0.2852

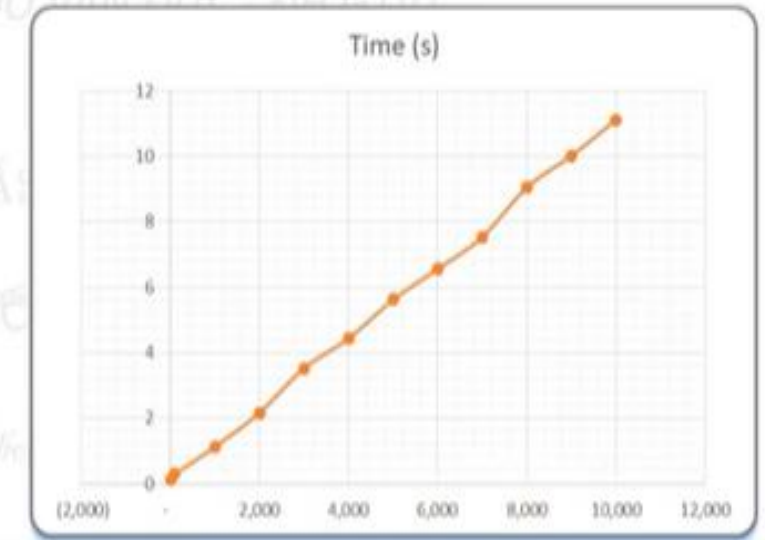
Thời gian chạy của chương trình mỗi lần là khác nhau phụ thuộc vào thời điểm thực thi và môi trường phần cứng, cũng như môi trường phần mềm

Ví dụ 2: Biểu diễn Thời gian chạy thuật toán (sử dụng 1 vòng lặp)

BT_04_01.cpp

```
1 #include <iostream>
2 #include <stdlib.h>
3 using namespace std;
4
5 int main(){
6     system("color F0");
7     int n = 10;
8     cout<<"\nBắt đầu chương trình\n";
9     for(int i = 0; i < n; i++)
10     {
11         int a = rand();
12         cout<< a<<" ";
13     }
14     cout<<"\nChương trình sinh "<<n<<" số ngẫu nhiên\n";
15     return 0;
16 }
```

N	Time (s)
10	0.1363
100	0.2919
1,000	1.129
2,000	2.172
3,000	3.527
4,000	4.44
5,000	5.624
6,000	6.561
7,000	7.539
8,000	9.052
9,000	10.02
10,000	11.12

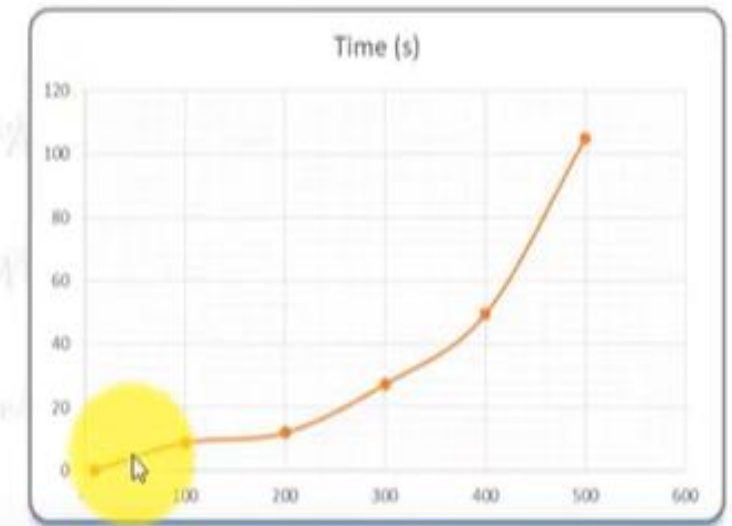


Ví dụ 3: Biểu diễn thời gian chạy của thuật toán với thuật toán khác (sử dụng 2 vòng lặp lồng nhau)

[*] BT_04_02.cpp

```
1 #include <iostream>
2 #include <stdlib.h>
3 using namespace std;
4
5 int main(){
6     system("color F0");
7     int n = 500;
8     for(int i = 0; i < n; i++)
9     for(int j = 0; j < n; j++)
10    {
11        int a = rand();
12        cout<<a<<" ";
13    }
14    cout<<"\n n = "<<n<<":\n";
15    return 0;
16 }
17 }
```

N	Time (s)
10	0.2169
100	8.786
200	11.9
300	27.26
400	49.37
500	104.9



Hạn chế của phương pháp thực nghiệm

1. Cần phải cài đặt thuật toán bằng một ngôn ngữ lập trình.
2. Phụ thuộc vào phần cứng và môi trường phần mềm mà thuật toán được cài đặt.
3. Với những bộ dữ liệu đầu vào chưa được thực nghiệm, không thể khẳng định chính xác.



1.5.

2.2 Phương pháp phân tích lý thuyết

1. Mô tả thuật toán bằng giả mã.
2. Xác định các phép toán cơ sở
3. Ước lượng thời gian chạy của thuật toán
4. Đánh giá thuật toán (thời gian chạy và dung lượng bộ nhớ cần thực hiện)



1.5. 3. Các giả thiết Phương pháp phân tích lý thuyết

Mô hình máy truy nhập ngẫu nhiên
(Random Access Machine (RAM) Model)

1. Một CPU
2. Không giới hạn số ô nhớ
3. Mỗi ô nhớ có thể lưu một số nguyên hoặc 1 ký tự
4. Mỗi ô nhớ được đánh số thứ tự.
5. Thời gian để truy cập đến mỗi ô nhớ được coi là như nhau, đó là một đơn vị thời gian tiêu chuẩn.

1.5. 4. Giả mã (Pseudocode)

Các ký hiệu:

← Phép gán giá trị,
= phép so sánh bằng,
Cho phép viết số mũ n^2 ,
Xuống dòng thay cho dấu {, }

Câu lệnh điều khiển:

If then else,
while do,
For do

Khai báo định nghĩa hàm, phương thức

Algorithm *hàm(dsdối)*

Input:

output:

Trả lại giá trị cho hàm: return Biểu thức



1.5.

5. Các phép toán cơ sở và cách xác định

Thời gian thực hiện các phép toán cơ sở được được xem là như nhau và độc lập với ngôn ngữ lập trình.

Các phép toán cơ sở:

1. Định giá một biểu thức
2. Gán giá trị cho một biến
3. Truy cập một phần tử mảng
4. Gọi hàm
5. Trả lại giá trị cho hàm (return)

Cách xác định số phép toán cơ sở

Bằng cách duyệt thuật toán:

⇒ xác định được số phép toán cơ sở tối đa của thuật toán.

⇒ xây dựng được một hàm thể hiện thời gian chạy của thuật toán.



Ví dụ 4: Ước lượng thời gian chạy của thuật toán?

Algorithm ArrayMax(A,n)

Input: n, Mảng A có n số nguyên

Output: Max, Giá trị lớn nhất của A

Max $\leftarrow$ A[0]

2

for i $\leftarrow$ 1 to n-1 do

n + 2

if A[i] > Max then

2(n-1)

Max $\leftarrow$ A[i]

2(n-1)

return Max

1

Tổng số phép toán cơ sở

5n + 1

Số phép toán
cơ sở

Ước lượng thời gian chạy

Thuật toán ArrayMax thực hiện $5n+1$ phép tính cơ bản trong trường hợp xấu nhất

Ký hiệu về thời gian thực hiện một phép toán cơ sở

a = Khoảng thời gian ngắn nhất

b = Khoảng thời gian dài nhất

$T(n)$ = là thời gian chạy của thuật toán trong trường hợp xấu nhất.

Ta có: $a(5n+1) < T(n) < b(5n+1)$

$\Rightarrow T(n)$ được bao bởi 2 đường tuyến tính

Ví dụ 4: Ước lượng thời gian chạy của thuật toán?

Nhận xét về Tốc độ tăng trưởng thời gian chạy của thuật toán

Khi thay đổi Phần cứng/Môi trường phần mềm,

⇒ a, b thay đổi ⇒ $T(n)$ thay đổi là 1 hằng số.

⇒ tốc độ tăng trưởng của $T(n)$ không thay đổi.

Tốc độ tăng trưởng tuyến tính của $T(n)$ là bản chất của thuật toán Arraymax.

Ước lượng thời gian chạy

Thuật toán ArrayMax thực hiện $5n+1$ phép tính cơ bản trong trường hợp xấu nhất

Ký hiệu về thời gian thực hiện một phép toán cơ sở

a = Khoảng thời gian ngắn nhất

b = Khoảng thời gian dài nhất

$T(n)$ = là thời gian chạy của thuật toán trong trường hợp xấu nhất.

Ta có: $a(5n+1) < T(n) < b(5n+1)$

⇒ $T(n)$ được bao bởi 2 đường tuyến tính

1.5. 6. Ký hiệu Ô-lớn (Big-Oh)

Định nghĩa Ô-lớn: Cho hàm $f(n)$ và $g(n)$, $f(n)$ được gọi là có ô lớn $g(n)$, kí hiệu là $O(g(n))$, nếu tồn tại hằng số dương c và số nguyên n_0 sao cho $f(n) \leq cg(n)$ với mọi $n \geq n_0$

Ví dụ 1:

$2n + 10$ là $O(n)$,

bởi vì nếu chọn $c = 3$ và $n_0 = 10$ thì $2n + 10 \leq 3n$ (tức $cg(n)$) luôn thỏa với mọi $n \geq 10$

Ví dụ 2:

$3n^3 + 20n^2 + 5$ là $O(n^3)$,

bởi vì nếu chọn $c = 4$ và $n_0 = 21$ thì $3n^3 + 20n^2 + 5 \leq 4n^3$ (tức $cg(n)$) luôn thỏa $\forall n \geq 21$

1.5.

6. Ký hiệu Ô-lớn (Big-Oh)

Định nghĩa Ô-lớn: Cho hàm $f(n)$ và $g(n)$, $f(n)$ được gọi là có ô lớn $g(n)$, kí hiệu là $O(g(n))$, nếu tồn tại hằng số dương c và số nguyên n_0 sao cho $f(n) \leq cg(n)$ với mọi $n \geq n_0$

Ô-lớn và tốc độ tăng trưởng giá trị

- Ký hiệu Ô-lớn chỉ ra một cận trên của tốc độ tăng trưởng giá trị của một hàm
- Ta nói “ $f(n)$ có ô lớn $g(n)$ ” có nghĩa là tốc độ tăng trưởng giá trị của $f(n)$ không lớn hơn tốc độ tăng trưởng giá trị của $g(n)$.

Ví dụ 1: $2n + 10$ là $O(n)$, có nghĩa là tốc độ tăng trưởng giá trị của $2n + 10$ không lớn hơn tốc độ tăng trưởng giá trị của n .

Ví dụ 2: $3n^3 + 20n^2 + 5$ là $O(n^3)$, có nghĩa là tốc độ tăng trưởng giá trị của $3n^3 + 20n^2 + 5$ không lớn hơn tốc độ tăng trưởng giá trị của n^3 .

1.5. 6. Ký hiệu Ô-lớn (Big-Oh)

Định nghĩa Ô-lớn: Cho hàm $f(n)$ và $g(n)$, $f(n)$ được gọi là có ô lớn $g(n)$, kí hiệu là $O(g(n))$, nếu tồn tại hằng số dương c và số nguyên n_0 sao cho $f(n) \leq cg(n)$ với mọi $n \geq n_0$

Qui tắc xác định Ô-lớn:

1. Nếu $f(n)$ là đa thức bậc d thì $f(n)$ là $O(n^d)$.
2. Sử dụng lớp hàm nhỏ nhất có thể
3. Sử dụng lớp hàm đơn giản nhất có thể

Ví dụ 1: $7n-2$ là $O(n)$

Ví dụ 2: $3n^3+20n^2+5$ là $O(n^3)$

Ví dụ 3: $3\log n + \log \log n$ là $O(\log n)$



1.5.

7. Đánh giá độ phức tạp của thuật toán

Đánh giá độ phức tạp của thuật toán chính là việc đánh giá thời gian chạy của thuật toán đó, nó được xác định bằng ký hiệu Ô-lớn (O)

Algorithm ArrayMax(A,n)

Input: n, Mảng A có n số nguyên

Output: Max, Giá trị lớn nhất của A

Số phép toán
cơ sở

Max $\leftarrow$ A[0]

2

for i $\leftarrow$ 1 to n-1 do

n + 2

if A[i] > Max then

2(n-1)

Max $\leftarrow$ A[i]

2(n-1)

return Max

1

Tổng số phép toán cơ sở

5n + 1

Thuật toán ArrayMax

có tối đa 5n+1 phép toán cơ bản

$\Rightarrow T(n) = 5n + 1$

$\Rightarrow$ Độ phức tạp của thuật toán là $O(n)$

1.5. 8. Các qui tắc cần thiết

Qui tắc cộng: Nếu một thuật toán thực hiện hai đoạn chương trình P1, P2 rời nhau và độ phức tạp của P1 là $O(g_1(n))$, của P2 là $O(g_2(n))$ thì độ phức tạp của thuật toán là $T(n) = O(\max\{g_1(n), g_2(n)\})$.

Qui tắc nhân: Nếu một thuật toán thực hiện hai đoạn chương trình P1, P2, trong đó P2 lồng trong P1 và độ phức tạp của P1 là $O(g_1(n))$, của P2 là $O(g_2(n))$ thì độ phức tạp của thuật toán là $T(n) = O(g_1(n) * g_2(n))$.



Ví dụ 5: Đánh giá độ phức tạp của thuật toán Min

Thuật toán Min

for i = 1 to n do input a[i];

Min ← a[0];

for i=1 to n-1 do

if a[i]<min then
min←a[i];

Thời gian chạy của thuật toán là T(n)

Đoạn chương trình

P1

P2

P3

Thời gian chạy

O(n)

O(1)

O(n)

Các đoạn
rời nhau

$T(n) = O(\max\{1, n, n\})$
 $= O(n)$

Ví dụ 6: Đánh giá độ phức tạp của thuật toán Sắp xếp

Thuật toán Sắp xếp	Đoạn chương trình	Thời gian chạy
for i = 1 to n do input a[i];	P1	$O(n)$
for i ← 1 to n-1 do	P2	$O(n)$
for j ← i+1 to n do		
if a[i]<a[j] then		
tg ← a[i];	P3	$O(n)$
a[i] ← a[j];		
a[j] ← tg;		

Thời gian chạy của thuật toán là $T(n)$

Đoạn chương trình P3 lồng trong P2 nên áp dụng quy tắc nhân → độ phức tạp của đoạn chương trình {P2 và P3} là: $O(n \cdot n) = O(n^2)$.

Đoạn P1 độc lập với {P2, P3} nên áp dụng quy tắc cộng cho đoạn chương trình gồm {P1, {P2, P3}} độ phức tạp của thuật toán: $T(n) = O(\max\{n, n^2\}) = O(n^2)$.



1.5. 9. Đánh giá thuật toán

Chính là việc Phân tích, đánh giá độ phức tạp (Thời gian chạy) và dung lượng bộ nhớ cần sử dụng của thuật toán

Algorithm arrayMax(A,n)

Input: n, Mảng a có n số nguyên

Output: Max, Giá trị lớn nhất của A

Max $\leftarrow$ a[0]

for i $\leftarrow$ 1 to n-1 do
if A[i] > Max then

Max $\leftarrow$ a[i]

return Max

các biến

n, a[0], ..., a[n-1]

max

i

Số biến

n + 2

1

Tổng số biến

N + 3

Bộ nhớ cần sử dụng (đơn vị bộ nhớ)

n + 3

thời gian chạy

O(n)



Ví dụ 7: Phân tích và đánh giá độ phức tạp của thuật toán sau

Cho dãy số thực $x[0], x[1], \dots, x[n-1]$.

Hãy xây dựng mảng $a[0], a[1], \dots, a[i-1]$ với $a[i] = (x[0] + x[1] + \dots + x[i]) / ((i+1))$.

Algorithm prefixAverage_1(x, n)

Input: n, mảng x có n số nguyên

Output: mảng a có n số nguyên

$a \leftarrow \text{new float}[n];$

for i $\leftarrow$ 0 **to** n-1 **do**

$s \leftarrow x[0];$

for j $\leftarrow$ 1 **to** i **do**

$s \leftarrow s + x[j];$

$a[i] \leftarrow s / (i+1);$

return a;

Số phép toán

Số biến

$n + 1$

n

n

$n + 3$

1

$2n$

1

$1+2+\dots+(n-1) + n + n$

1

$3(1+2+\dots+(n-1))$

$4n$

1

$T_1(n) = 2n^2 + 8n + 4$

$2n + 4$

$2n + 4$

Tổng

Bộ nhớ

Thời gian chạy $O(n^2)$

Ví dụ 8: Phân tích và đánh giá độ phức tạp của thuật toán sau

Cho dãy số thực $x[0], x[1], \dots, x[n-1]$.

Hãy xây dựng mảng $a[0], a[1], \dots, a[i-1]$ với $a[i] = (x[0] + x[1] + \dots + x[i]) / ((i+1))$.

Algorithm `prefxAverage_2(x, n)`

Input: n , mảng x có n số nguyên

Output: mảng a có n số nguyên

$a \leftarrow \text{new float}[n];$

$s \leftarrow 0;$

for $i \leftarrow 0$ **to** $n-1$ **do**

$s \leftarrow s + x[i];$

$a[i] \leftarrow s / (i+1);$

return $a;$

Tổng

Bộ nhớ

thời gian chạy

Số phép toán

Số biến

$n + 1$
 n

n
 1

1
 1

$3n$

$4n$

1

$T_2(n) = 9n + 5$

$2n + 3$

$2n + 4$

$O(n)$

Nhận xét về Ví dụ 7 & 8

Cho dãy số thực $x[0], x[1], \dots, x[n-1]$.

Hãy xây dựng mảng $a[0], a[1], \dots, a[i-1]$ với $a[i] = (x[0] + x[1] + \dots + x[i]) / ((i+1))$.

Algorithm prefixAverage_2(x, n)

Input: n, mảng x có n số nguyên

Output: mảng a có n số nguyên

```
a ← new float[n];  
s ← 0;  
for i ← 0 to n-1 do  
    s ← s + x[i];  
    a[i] ← s/(i+1);  
return a;
```

$$T_2(n) = 9n + 5$$

Hãy nghĩ đến thuật toán tốt để giải một bài toán

Algorithm prefixAverage_1(x, n)

Input: n, mảng x có n số nguyên

Output: mảng a có n số nguyên

```
a ← new float[n];  
for i ← 0 to n-1 do  
    s ← x[0];  
    for j ← 1 to i do  
        s ← s + x[j];  
    a[i] ← s/(i+1);  
return a;
```

$$T_1(n) = 2n^2 + 8n + 4$$

